# GRAFT: Cross-Engine Bug-Report-Guided Fuzzing of a SQLite-Compatible Database

ANONYMOUS AUTHOR(S)

A database engine must answer every statement or reject it with an error. Mature engines check this with differential fuzzers that compare their answers against a reference engine. Such a fuzzer cannot test a function it never generates, nor judge one its reference engine lacks. Turso, a SQLite-compatible engine written in Rust, runs such a fuzzer in continuous integration, yet the single statement `select LPAD('a', 9223372036854775807, 'b')` terminated the whole process. We present Graft. It takes the shape of bug reports filed against a different engine, DuckDB, refills their literals with boundary values, and runs them against Turso together with a typed SQL generator. Its only oracle needs no reference engine: one statement must not end the process. Graft found three previously unknown defects in Turso's `lpad`, `rpad` and `repeat` functions. Each aborts the engine from one statement, and Graft regenerates each from a recorded seed. Our patch for them exposed a fourth defect in the same length check, which Graft did not find. Our two patches, which fix all four, were merged upstream. None of Turso's own fuzzers can express any of the four, since the three functions are absent from all of them. At an equal budget of 20,000 cases, mined reports and typed generation each reached both root causes of the three defects, and drawing from both sources found the first defect at case 101. The work taught us that reading an incumbent fuzzer's source is a cheap and reliable way to decide where to test next.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Data management systems.*

Additional Key Words and Phrases: fuzzing, database testing, SQL, bug reports, test generation, crash oracle, recurring defects

**ACM Reference Format:**
Anonymous Author(s). 2026. GRAFT: Cross-Engine Bug-Report-Guided Fuzzing of a SQLite-Compatible Database. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (Format Template) (FSE (CS-5187 Term Project))*. ACM, New York, NY, USA, 14 pages.

## 1 Motivation

Suppose a server process embeds a SQL engine and lets its users format reports with `lpad`, which pads a string to a requested width. A user types a width of 9,223,372,036,854,775,807. The server should answer with an error message. If the engine instead ends the process, every other user's session ends with it. A database engine is trusted to be correct without being checked, so a defect of this kind is caught by the engine's own tests or not at all.

Database engines are among the most heavily tested programs in practice. Differential testing runs the same query on two engines and reports any disagreement. Metamorphic oracles compare a query with an equivalent rewrite of itself [Rigger and Su 2020a,b]. Grammar-based generators produce valid SQL at scale [Liang and Liu 2026; Zhong et al. 2020]. All of these tools stop at two boundaries: the generator can only emit the functions and values it was written to emit, and a differential oracle can only judge a feature that its reference engine also implements. We focus on the defects that sit past those two boundaries. Such defects are silent in a specific way: a fuzzer can run for a year without reaching them, and the time it has already run says nothing about how close it is.

Figure 1 shows a real case. Turso [Turso Database 2026b] is a SQLite-compatible engine written in Rust. It runs its own differential fuzzer in continuous integration: 41,205 lines of Rust that generate SQL and compare Turso's answers with SQLite's. Yet the statement in Figure 1d ended the process

*FSE (CS-5187 Term Project), IIT Guwahati*
2026.

```
-- (a) Regression test added by the fix for DuckDB issue #27846
select LPAD('x', 10, '0'), LPAD('x', 4, '★'),
       LPAD('x', 3, '★ab'), RPAD('x', 3, '★ab')
-- (b) The same shape with every literal replaced by a typed hole
select LPAD({s4}, {n0}, {s5}), LPAD({s6}, {n1}, {s7}),
       LPAD({s8}, {n2}, {s9}), RPAD({s10}, {n3}, {s11})
-- (c) Filled from a boundary pool at seed 0, then run on Turso b588fbdc6
select LPAD('', -9223372036854775808, 'abc'), LPAD('_', -128, '%'),
       LPAD('0', 9223372036854775807, 'abc'), RPAD('%', -128, 'ABC')
-- (d) Reduced to one statement
select LPAD('a', 9223372036854775807, 'b');
memory allocation of 9223372036854775806 bytes failed      ← process aborts, exit status 134
```

Fig. 1. The motivating example. A regression test from another engine (a), templated (b) and refilled with boundary values (c), aborts Turso from one statement (d). The marker ★ stands for a four-byte emoji in the original test. Turso's own fuzzers cannot produce (d): their 47-function vocabulary has no `lpad`, and their integers stop at 1,000,000.

with exit status 134 (SIGABRT) in 2.28 seconds, where it should have returned the engine's own error *string or blob too big*. Turso's fuzzer could never have produced it. Its vocabulary of 47 scalar functions does not contain `lpad`, and it draws integer literals only from [−1,000,000, 1,000,000] [Turso Database 2026a]. Even if it had produced the statement, SQLite has no `lpad` to compare the answer against. This is not a single case: Graft found three such defects, and none of Turso's four libFuzzer targets or its differential fuzzer calls any of the three functions involved.

The closest prior work either chooses where to look from the grammar or reuses known defects through static analysis. SQLBull starts generation from grammar rules that a tester marks as feature-rich [Liang and Liu 2026]. Recurring-vulnerability detectors turn a known patch into a rule and search for code that breaks it [Cao et al. 2025a,b]. However, the first needs someone to pick the starting rules, and the second needs the defect to recur in shared or copied code. Turso and DuckDB share no code.

We present Graft, which finds defects in this region in four steps:

(1) It reads the incumbent fuzzer's source and lists the functions and values that fuzzer cannot generate (Section 2.4).
(2) It mines closed, reproduced bug reports from a different engine and keeps each report's query as a template whose literals are typed holes (Section 2.5).
(3) It fills those holes, and the output of a typed SQL generator, with boundary values, and runs every statement against the engine under test (Section 2.6).
(4) It flags any statement that ends the process, then reduces, re-checks and reports it (Section 2.7).

We ran Graft against Turso. It found three previously unknown defects in Turso's string functions: `lpad` and `rpad` allocate their result before checking its length, and `repeat` also overflows its size computation. Our patch for them moved the length check in front of the allocation, and that exposed a fourth defect, which Graft did not find: the check counts characters against a limit in bytes. Our two patches, which fix all four, were merged upstream [Anonymous 2026d,e]. At an equal budget of 20,000 cases, the mined reports and the typed generator each reached both root causes of the three defects. The lesson we draw is a practical one: before spending compute on a fuzzer, spend an hour reading the fuzzer that is already there.

*Contributions.*

- We define the *blind spot* of an incumbent fuzzer and show that it can be measured from that fuzzer's source (Section 2.2).
- We present Graft, which fills the blind spot with the shapes of another engine's bug reports and with typed generation, and judges results with an oracle that needs no reference engine (Section 2).
- We report three previously unknown defects in Turso found by Graft, each regenerated from a recorded seed, and a fourth that our fix exposed. All four are fixed upstream by our merged patches (Section 4.5).
- We measure what each input source contributes at an equal budget, and release the tool, the corpus and all data (Section 4.6 and the Data Availability statement).

Section 2 gives the background the paper needs, states the problem precisely and presents Graft. Section 3 describes its implementation, Section 4 evaluates it, and Sections 5 and 6 discuss related work and the threats to validity.

### 1.1 Why the Obvious Alternatives Fall Short

*Why not widen the incumbent fuzzer instead?* Its maintainers could add `lpad` to its function vocabulary and `i64::MAX` to its literal domain, and the defects in this paper would then be within reach. But that needs someone to know which function and which value to add. The blind spot is large: any function outside the vocabulary and any value outside the literal domain belongs to it (Section 2.2 defines it formally). Bug reports from another engine point at the corners where someone has already been hurt. A crash oracle then removes the need for a reference engine, so a function SQLite lacks can still be tested.

*Why not run the other engine's tests directly?* A regression test was written for the other engine's defect. Its literal values are tuned to that defect, which the engine under test probably never had, and its syntax may not parse. The structure of the query, meaning which functions are nested how, is what carries over. The values have to be chosen again.

## 2 Technique

This section first gives the background the paper needs (Section 2.1) and defines the problem (Section 2.2). It then describes the workflow of Graft and its four steps.

### 2.1 Background

*Turso.* Turso [Turso Database 2026b] reimplements SQLite in Rust and aims to accept the same SQL. Its shell, `tursodb`, reads statements on standard input. Text and blob results are bounded by a constant, `Value::MAX_BLOB_LENGTH`, of $10^9$ bytes. A statement whose result would exceed it should fail with the error *string or blob too big*, which is SQLite error code 18. Rust aborts the process when an allocation fails and cannot be recovered, so a size check that runs after the allocation is not a check at all.

*Differential testing.* A *differential oracle* runs one query on two engines and reports a disagreement. It needs no knowledge of the correct answer, but it needs a *reference engine* that implements the same feature. Turso's fuzzer uses SQLite as its reference engine. SQLite has no `lpad`, `rpad` or `repeat`, so a disagreement on those functions cannot be judged.

*Crash oracle.* A *crash oracle* reports a statement that ends the engine's process, panics, reports an internal error or does not answer within a time limit. It does not report a *user error*, which is a statement the engine rejects with a normal message, such as a syntax error or a documented limit. A crash oracle needs no reference engine. It finds no wrong answer that leaves the process running.

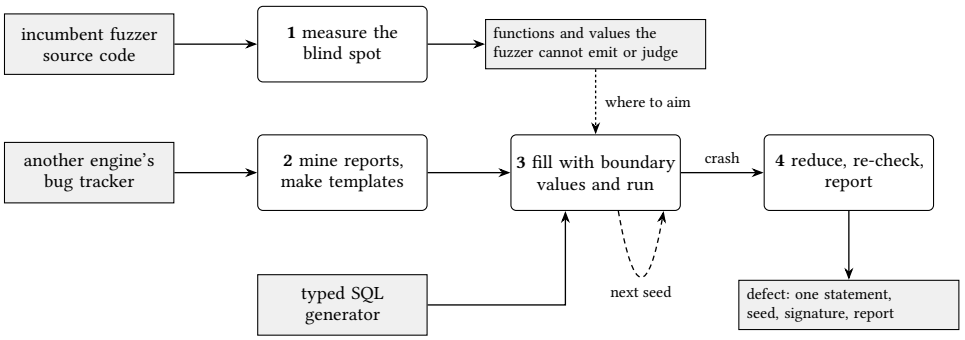



Fig. 2. Workflow of Graft. Step 1 runs once and tells Step 3 where to aim. Steps 2 and 3 supply two sources of statements, mined templates and a typed generator. Step 3 loops over seeds and is the only step whose cost grows with the budget; a crash goes to Step 4.

*Bug reports as test material.* DuckDB maintainers add the label *reproduced* to an issue once they have confirmed it. A fix usually adds a regression test that contains the query which triggered the issue. Such a query is small, it was chosen by someone who understood the defect, and it tends to sit in an awkward corner: an empty string, a multi-byte character, a value at the edge of a type.

*Literal and template.* We call a constant in a query, such as `10` or `'x'`, a *literal*. A *template* is a query in which every literal is replaced by a typed hole, such as `{n0}` for an integer or `{s4}` for a string. Filling the holes gives a new query with the same structure.

## 2.2 Problem Definition: The Blind Spot of an Incumbent Fuzzer

Let $F$ be the fuzzer that already tests an engine $E$, and let $R$ be the reference engine that $F$ compares against. We write $V_F$ for the set of SQL functions that $F$ can emit, and $L_F$ for the literals it can emit: a range of integers and a set of strings. We write $V_R$ for the functions that $R$ implements.

A statement $q$ is *expressible* by $F$ if every function in $q$ is in $V_F$ and every literal in $q$ is in $L_F$. A statement is *judgeable* if every function in $q$ is in $V_R$. The *blind spot* of $F$ is

$$B = \{\, q \mid q \text{ is not expressible by } F \ \vee \ q \text{ is not judgeable against } R \,\}. \tag{1}$$

$F$ never generates a statement outside its own vocabulary or literal domain, however long it runs. A statement that $F$ does generate but cannot judge is generated in vain. Both halves of (1) are properties of the source code of $F$ and $R$, so $B$ can be measured before any test runs.

*Goal.* Find statements $q \in B$ on which $E$ violates the crash oracle, without a reference engine, and with each finding regenerated from a recorded seed. We look for two things: which functions and values put a statement in $B$, and where to get statements in that region that are likely to break $E$.

## 2.3 Workflow

Figure 2 shows the workflow of Graft. The main idea is to aim testing at the blind spot of the incumbent fuzzer, and to take the statements that reach it from bug reports another engine's maintainers have already reduced. Two kinds of input go in: the incumbent fuzzer's source, and a bug tracker. One kind comes out: a statement that ends the engine's process, with the seed that regenerates it. Step 3 loops over seeds and is the only expensive step.

### 2.4 Step 1: Measure the Blind Spot

We read the incumbent fuzzer's source and extract three facts: (i) its function vocabulary $V_F$; (ii) its literal domain $L_F$; and (iii) its reference engine $R$. For Turso these are a table of 47 scalar functions and a default literal policy in two source files [Turso Database 2026a]. The policy sets `int_max` to 1,000,000 and restricts strings to alphanumeric characters of at most 50 bytes. Every function outside $V_F$, every integer outside the range, every non-alphanumeric string, and every function SQLite lacks is in $B$ by Equation (1). This step takes reading, not running. It needs no new tool, and we do not claim reading source code as new. What we claim is that this reading should come first.

### 2.5 Step 2: Mine Reports and Make Templates

Graft collects closed issues that carry the tracker's label for a confirmed defect; for DuckDB this label is *reproduced*. For each issue it fetches the fixing commit and the regression test that the fix added, and extracts every SQL statement from that test and from the issue body. Setup statements such as `CREATE TABLE` and `INSERT` are kept with their query, so a reproduction brings its own schema and data.

Each extracted query becomes a template. Graft replaces every literal with a hole typed by the literal's kind (integer, real or string) and leaves everything else exactly as written: function names, nesting, clauses and setup. Figure 1b shows the result for one query. We keep the structure fixed on purpose. Re-pointing a mined query at a new schema would need column types and arities to match, and a wrong substitution produces SQL that merely looks plausible. A query whose structure is unchanged is valid by construction, so no repair step is needed. The cost is that Graft cannot explore a structure that no report contains.

### 2.6 Step 3: Fill with Boundary Values and Run

Graft fills each hole from a pool weighted toward the edges of each type. For integers the pool holds zero, ±1, the limits of 8-, 16- and 32-bit integers, and the limits of 64-bit integers, including `i64::MAX`. String holes are filled from a pool of short ASCII strings. Every random choice comes from a generator seeded per case, and the seed is stored with each result.

Mined templates are one source of statements. The second is a typed SQL generator that builds a schema, inserts rows, and composes expressions and queries. It emits only features that the target engine accepted when Graft probed it, so it rarely wastes a case on syntax the engine rejects. Its string pool also includes multi-byte values such as `'é'` and three-byte Chinese characters. A command line flag selects either source or both.

Each case, meaning its setup statements and its query, runs in a fresh `tursodb` process under a time limit and a memory limit. The crash oracle classifies the outcome into one of six classes (*none*, *user error*, *internal error*, *panic*, *crash*, *timeout*) and reports only the last four. We do not claim the crash oracle as new; fuzzers have used it for decades. We chose it because it is the one oracle that can judge a function the reference engine lacks.

### 2.7 Step 4: Reduce, Re-check and Report

A violation is not yet a finding. Graft groups violations by a *signature*, a hash of the oracle, the error class and the error text with numbers and paths removed. It then (i) reduces the statement while keeping the same violation; (ii) re-runs it on a fresh process and on a build of the current upstream main branch, so that a defect fixed last week is not reported; (iii) searches the upstream tracker and records the queries it used; and (iv) drafts a report. A person reads the draft, reproduces it, and decides whether to file it. Reduction by repeated deletion is standard and we do not claim it

as new. The duplicate check matters more than it looks. In our campaign it withheld one candidate, because an upstream pull request had already changed the code it touched.

## 3 Implementation

Graft is written in Python 3.14 in 10,034 lines across 38 files, grouped into five packages: engine adapters, the corpus pipeline, the typed generator, the oracle, and triage. It depends on four libraries: PyYAML for configuration, requests for the GitHub API, sqlglot for reducing statements that parse, and pytest. A suite of 514 tests (5,041 lines) passes in about 31 seconds. Two problems we met while building it are worth reporting, because both produced wrong results that looked like right ones.

*Errors arrive in the middle of results.* Graft drives `tursodb` as a subprocess rather than through a library binding. The defects we look for end the process, and an in-process binding would take the test harness down with it. The cost is a text protocol. Turso writes errors to standard output, interleaved with result rows. Our first adapter looked for an error only on the first line of the output. A query that returned two rows and then failed therefore came back as three rows of data, the third being the error message. The adapter now scans the whole output, and a regression test covers this case.

*A declared type is not a stored type.* The generator first assumed that if `CREATE TABLE t(c DATE)` parsed, the engine supported dates. SQLite and Turso map any unknown type name to a storage class, so such a column really holds integers. A generator built on that assumption reports type coverage it does not have. Graft therefore probes the real binary once, records which types it can tell apart and which functions exist, and refuses to emit anything else.

*The seed is the state.* Every random choice comes from a seeded generator, and a campaign writes its last seed on every progress report. A campaign can stop and resume exactly, and any single case can be regenerated from its seed alone. Section 4.5 uses this to regenerate each defect.

## 4 Evaluation

We ask four questions.

**RQ1** Does Graft find previously unknown defects in an engine that already fuzzes itself, and are they accepted upstream?

**RQ2** At an equal budget, which defects does each source of statements reach: mined templates, the typed generator, or both?

**RQ3** Do the defects lie in the blind spot of the engine's own fuzzers, or could those fuzzers have found them with more time?

**RQ4** What does Graft cost in time and in requests?

### 4.1 Subjects

We test one engine and mine one other. We chose the engine under test with the following criteria:

(1) it is actively maintained and reviews outside contributions, so a report gets an answer within weeks;
(2) it can be built from source at any commit, so a defect can be confirmed on the current main branch and shown fixed afterwards;
(3) it already runs its own fuzzers, so a defect we find is one those fuzzers missed rather than one nobody looked for.

Turso meets all three. We used version 0.9.0-pre.2 at four commits: 730017f96, where the first campaign ran; b588fbdc6, the last commit before our fixes; ec86571642a6, which has only our first patch; and 7f0a9e6f4, which has both. We mined DuckDB because it labels confirmed issues and

Table 1. The four defects, each run as one statement in a fresh process. *Before*: Turso b588fbdc6, which has none of our patches. *After*: 7f0a9e6f4, which has both. Exit status 134 is SIGABRT. *Found by*: how the defect was first found. *Seed*: the seed that makes Graft regenerate the defect in a single case in the run that draws from both sources. Defect 4 ran under a 6 GB memory limit; 3,999,999,996 is $10^9$ characters times 4 bytes, minus the one-character input.

| | Function | Defect | Before | After | Found by | Seed |
|---|---|---|---|---|---|---|
| 1 | `lpad` | allocates before the length check | 134, 2.28 s | 1, 0.02 s | Graft | 4,553 |
| 2 | `rpad` | allocates before the length check | 134, 2.24 s | 1, 0.02 s | Graft | 101 |
| 3 | `repeat` | allocates first; size can overflow | 134, 2.27 s | 1, 0.02 s | Graft | 5,080 |
| 4 | `lpad` | counts characters, limit is bytes | 134, 3.96 s | 1, 0.02 s | review of our patch | – |

because its fixes usually carry a regression test with the triggering query [DuckDB 2026]. From 600 closed issues Graft kept 200, and from those it extracted 659 reproductions: 526 from regression tests and 133 from issue bodies.

### 4.2 Baselines

**Mined only.** Graft with templates from DuckDB reports and no generator.
**Generated only.** Graft with the typed generator and no templates. This is the plain baseline: a grammar-based fuzzer with boundary values.
**Both.** Each case is drawn from either source.
**Turso's own fuzzers.** Not run by us. We read their source to answer **RQ3**.

All three runs use the same budget of 20,000 cases, the same seeds 0 to 19,999, the same crash oracle, and the same binary, b588fbdc6, which has none of our patches.

### 4.3 Measurements

**RQ1.** Defects found, their upstream state, the exit status and output of each reduced statement before and after our patches, and whether the tool regenerates each one from a single seed.
**RQ2.** Per run: cases executed, the share the engine accepted, oracle hits, and distinct signatures, with the first seed that reaches each. The same runs repeated on the build with only our first patch.
**RQ3.** For each defect, whether its function is in Turso's fuzzer vocabulary, whether its literals are in that fuzzer's domain, and whether SQLite implements the function.
**RQ4.** Wall-clock time and cases per second for each run, and requests made to build the corpus.

### 4.4 Setup

All measurements ran on one laptop: an Intel Core i7-1360P with 16 logical CPUs and 15 GB of memory, under Linux kernel 7.0, with Python 3.14.4 and Rust 1.88.0. Each case had a 10-second time limit. The statement for defect 4 also ran under a 6 GB memory limit, so the pre-fix build would fail fast instead of building a string of several gigabytes. Each run in Section 4.6 was done once; Section 6 discusses what that allows us to claim.

### 4.5 RQ1: Does Graft Find New Defects?

Table 1 lists the defects. Defects 1 to 3 share a cause: `lpad`, `rpad` and `repeat` sized their result from the caller's argument and allocated it before comparing it with `Value::MAX_BLOB_LENGTH`. At a length near `i64::MAX` the allocation fails and Rust aborts the process before the check, which ran

Table 2. Each source of statements at an equal budget of 20,000 cases and seeds 0 to 19,999. *Valid*: share of executed cases the engine accepted. *Hits*: oracle violations on the pre-fix build. *Distinct*: distinct signatures, the unit of comparison, since eight hits on one signature are one defect found eight times. The next three columns give the first seed that reached each signature, or a dash if none did. Defects 1 and 2 share a signature; in all three runs its first hit was an `rpad` call. The third signature, *Defect 1 again*, is defect 1 reached with a length above $10^9$ computed from table data. *After patch 1*: hits when the same run is repeated on ec86571642a6, the build with only our first patch.

| Source | Executed | Valid | Hits | Distinct | Defects 1, 2 | Defect 3 | Defect 1 again | After patch 1 |
|---|---|---|---|---|---|---|---|---|
| Mined only | 19,169 | 8.9% | 8 | 2 | 914 | 10,180 | – | 0 |
| Generated only | 20,000 | 98.3% | 12 | 3 | 762 | 5,367 | 6,376 | 0 |
| Both | 19,759 | 71.1% | 15 | 3 | 101 | 5,080 | 5,429 | 0 |

only on the finished result, is reached. `lpad` and `rpad` share one function body, `pad()`, so defects 1 and 2 have one root cause. We count them separately because each aborts the engine on its own, and a user who calls only `rpad` is exposed. `repeat` is a separate function that also multiplies, so its fix needed an overflow check as well. Graft found all three: the first abort came eight seconds into the first campaign. One patch fixed them [Anonymous 2026b,d].

Defect 4 became reachable only after that patch, and Graft did not find it. With the check now running first, it compared a count of characters with a limit in bytes. A four-byte fill such as an emoji passes a check of $10^9$ characters, and about 4 GB is built before the byte limit rejects it. We noticed the unit mismatch while writing the first patch, but our pull request described it as "conservative", meaning harmless. Turso's automated reviewer replied that it was permissive and supplied a demonstrating input. We then diagnosed the root cause, and a second patch computes the exact byte length before building anything [Anonymous 2026c,e]. On a debug build without a memory limit, the statement ran for about 34 seconds and peaked at 11.7 GB.

**Observation 1.** Graft found three previously unknown defects in Turso, each of which ends the process from one statement. Fixing them exposed a fourth, which Graft did not find. Our two patches, which fix all four, were merged upstream.

**Observation 2.** Graft regenerates each of the three defects it found from one recorded seed, in a single case, on the pre-fix build: seed 4,553 for defect 1, 101 for defect 2 and 5,080 for defect 3. We checked that each seed calls the function its defect names.

### 4.6 RQ2: Mined Templates versus Typed Generation

Table 2 gives the result. Both sources reached both root causes of defects 1 to 3: the shared `pad()` body and `repeat`. The generator reached one more signature. On inspection it is defect 1 again: an `lpad` call whose length, above $10^9$, is computed from table data. We first took this signature for defect 4. Repeating every run on the build with only our first patch settled it: all hits disappear, while that build still aborts on the defect-4 statement. So no source reached defect 4.

The reason is in the boundary pools. Defect 4 needs a length of at most $10^9$ characters whose result is more than $10^9$ bytes, so between $2.5 \times 10^8$ and $10^9$ for a four-byte fill. The integer pools jump from 32,767 to 2,147,483,647. None of the 60,000 cases that the three modes generate for seeds 0 to 19,999 contains an integer literal in that range. The pools hold the limits of machine integers, not the limits the engine sets itself.

Mined templates are wasteful but well aimed. Only 8.9% of their cases ran at all, because most DuckDB reproductions use syntax Turso rejects. Still, those few valid cases reached both root

Table 3. Why Turso's own fuzzers cannot find the four defects, measured on the source at b588fbdc6 [Turso Database 2026a]. *In vocabulary*: the function is among the 47 scalar functions of the differential fuzzer, or among those its four libFuzzer targets call. *Literal in domain*: the defect's literal lies in the fuzzer's integer range $[-10^6, 10^6]$ or its alphanumeric string set. *Judgeable*: SQLite implements the function.

| | Function | Literal the defect needs | In vocabulary | Literal in domain | Judgeable |
|---|---|---|---|---|---|
| 1 | `lpad` | integer $9.2 \times 10^{18}$ | ✗ | ✗ | ✗ |
| 2 | `rpad` | integer $9.2 \times 10^{18}$ | ✗ | ✗ | ✗ |
| 3 | `repeat` | integer $9.2 \times 10^{18}$ | ✗ | ✗ | ✗ |
| 4 | `lpad` | a multi-byte fill character | ✗ | ✗ | ✗ |

Table 4. Cost of each 20,000-case run in Table 2, on the pre-fix build and the laptop described in the setup. *Time*: wall-clock seconds for the whole run. *Rate*: cases executed per second. *First defect*: the case number of the first oracle hit; the run that draws from both sources is the slowest per case and the fastest to its first defect.

| Source | Time (s) | Rate (cases/s) | First defect (case) |
|---|---|---|---|
| Mined only | 378.0 | 50.7 | 914 |
| Generated only | 388.8 | 51.4 | 762 |
| Both | 448.3 | 44.1 | 101 |

causes. Drawing from both sources found the first defect at seed 101, against 762 and 914 for either source alone, and recorded the most hits.

**Observation 3.** At an equal budget, mined templates and typed generation each reached both root causes of defects 1 to 3. No source reached defect 4, because no boundary pool holds a length near the engine's own limit of $10^9$.

**Observation 4.** Drawing from both sources reached the first defect at seed 101, compared with 762 and 914 for each source alone, and recorded 15 hits against 12 and 8.

### 4.7 RQ3: Could Turso's Own Fuzzers Have Found Them?

Table 3 answers the question with three independent checks. The differential fuzzer's table of 47 scalar functions contains none of `lpad`, `rpad` or `repeat`. None of its four libFuzzer targets, and no Rust file under its `testing` directory, mentions them either. The functions are covered only by hand-written unit tests with small values. Even if the functions were added, the default integer range stops at $10^6$, and defects 1 to 3 need a length near `i64::MAX`. Strings are alphanumeric, and defect 4 needs a multi-byte character. Finally, SQLite has none of the three functions, so a differential oracle would have nothing to compare against.

**Observation 5.** All four defects lie in the blind spot of Turso's own fuzzers on every axis we checked: the functions are absent from their vocabulary, the required literals are outside their domain, and the reference engine lacks the functions. More time would not have found them.

### 4.8 RQ4: What Does GRAFT Cost?

Table 4 gives the cost of each run. Every run finished in under eight minutes. Drawing from both sources was the slowest per case and the fastest to a defect: at 44.1 cases per second, case 101 is reached in about two seconds. We do not have an explanation for the lower rate, and with one run per source we cannot tell it apart from noise. Building the corpus took 387 requests to the GitHub

API, 354 of them answered from a local cache, and is done once. Step 1 costs reading time rather than compute. The reducer shrank the case for defect 3 to a single statement in five executions.

**Observation 6.** One run of 20,000 cases took under eight minutes on a laptop. With both sources, the first defect appeared at case 101, about two seconds into the run.

### 4.9 Discussion

*Is mining bug reports better than generating SQL?.* Not on this target. At the same budget both sources reached the same two root causes (Observation 3). Mining did two things the generator did not. It found defects 1 and 3 first in time, before the generator's vocabulary contained `lpad` or `repeat`; we added those functions to the generator only after the defects were filed. And it pointed the work at pad functions in the first place. The fair reading is that each source covers a weakness of the other, which is why drawing from both reached a defect soonest (Observation 4).

*Are the defects in one family really four?* They are four ways for a user to end the process, each from a different statement, so we count them as four. They are not four independent mistakes: defects 1 and 2 share one function body, and defects 1 to 3 share one cause and one patch. Upstream records them as two issues and two pull requests. We report both counts and say which is which, so that neither inflates the other.

*Would more fuzzing time have found them?* No. Observation 5 is a property of the fuzzer's source, not of its budget. A generator cannot emit a function outside its vocabulary or a value outside its domain however long it runs. This is the main practical point of the paper: the incumbent fuzzer's run time tells nothing about how close it is to these defects. The same holds for Graft and defect 4: its pools contain no length in the range that defect needs.

*Why did fixing three defects reveal a fourth?* Before the first patch, the length check ran after the allocation, so its unit did not matter: the process died first. Once the check ran first, its unit became the thing that decided the outcome. A patch is a good place to look for the next defect, since it marks code where the authors' first understanding was wrong. Our own first understanding of defect 4 was wrong too, and a reviewer corrected it.

*Does this work on other engines?* We do not know. The method needs a second engine with many closed, labelled reports that carry reproductions, and an engine under test that shares enough of its dialect for some reports to run. Only 16 of 60 reports that we classified by hand looked transferable to another engine. Testing other pairs of engines, filling string holes with multi-byte values, and adding each engine's own limits to the boundary pools we leave for future work.

## 5 Related Work

*Grammar-based SQL generation.* Closest to us is SQLBull, which starts generation from grammar rules a tester marks as feature-rich, then builds a path back to the root so that every query contains the rule; it found 63 previously unknown defects in five engines, all crashes, internal errors or assertion failures [Liang and Liu 2026]. SQUIRREL keeps mutated SQL valid and guides the search with coverage feedback [Zhong et al. 2020]. We share their goal and their kind of oracle, but we choose where to aim differently. SQLBull aims at deep grammar, a property of the language. We aim at what a specific incumbent fuzzer cannot express, a property of its source, and we take the queries from another engine's bug reports rather than from a grammar walk.

*Oracles for database engines.* Ternary logic partitioning splits a query into three partitions whose union must equal the original [Rigger and Su 2020b]. Non-optimizing reference engine construction

compares an optimized query with a rewrite the optimizer cannot touch [Rigger and Su 2020a]. Both find wrong answers that leave the process running, which a crash oracle cannot see. We use the weaker oracle on purpose: it needs neither a reference engine nor an equivalent rewrite, so it can judge `lpad`, which SQLite does not have.

*Keeping generated input valid.* Grammar-based whitebox fuzzing lifts symbolic constraints to tokens so that solved inputs parse [Godefroid et al. 2008]. Fuzzilli mutates an intermediate language in which every program stays valid [Groß et al. 2023]. Repair-driven greybox fuzzing mutates freely and repairs the result back to valid syntax [Bendrissou et al. 2026]. Directed grammar-based generation learns which input structures reach a testing goal from feedback [Kirschner and Soremekun 2025]. FuzzNG fuzzes Linux system calls without hand-written descriptions, and found defects in components that Syzkaller had covered for years [Bulekov et al. 2023]. We solve the validity problem more crudely: we keep a mined query's structure fixed and change only its literals, and we learn nothing from feedback. FuzzNG's lesson, that a long-running incumbent fuzzer still leaves reachable defects, holds here too; we differ in locating those defects by reading the incumbent's source rather than by reshaping its input space.

*Recurring defects.* VulPA writes a known vulnerability as a pattern and finds other code that breaks it with multi-object typestate analysis [Cao et al. 2025a]. FirmRec builds signatures of known vulnerabilities from how they are exploited and finds them again in firmware images [Xiao et al. 2024]. A recent study measures the errors of recurring-vulnerability detectors on vulnerabilities inherited through reused code [Cao et al. 2025b]. Another shows that code which looks like a known vulnerability but is already patched causes many false alarms [Tan et al. 2025]. We share the question these works ask: does a known defect have siblings? But they answer it statically, within one code base or across copied code. We answer it dynamically, across two engines that share no code. Our duplicate check before filing plays the role that a patched-code filter plays for them, but each of our answers is a statement that either ends the process or does not, so there is no false alarm to filter.

*Learning from patches.* Mystique ports a vulnerability patch from one branch to another with a language model [Wu et al. 2025], and VicDiff locates the commit that introduced a vulnerability by comparing patching patterns [Guo and He 2026]. Both reuse what a patch says about a defect. We reuse what a fix's regression test says about where defects live, and move a failing input rather than a fix.

## 6 Threats to Validity

*Internal threats.* A harness defect can look like an engine defect. During development our own adapter once read an error message as a result row (Section 3). We guarded against this as follows:

- every reported statement was re-run by hand on a fresh process, on the current upstream main branch, before it was filed;
- each defect was confirmed on two builds of the same source tree that differ only by our patches, so the patch explains the change;
- a test engine with planted defects checks that the oracle fires on a real defect and stays silent on a correct engine and on invalid SQL;
- every seed in Table 1 was run again in a single case and reproduced the signature claimed.

Attributing a hit to a defect is a second internal threat. We first mapped one signature to defect 4 by hand, and it was defect 1 reached another way (Section 4.6). Every attribution in this paper was therefore checked against the build that has only our first patch, which separates defect 4 from the other three. Each run in Table 2 was done once. Seeds make a run repeatable on the same code, so

the counts there are exact for that code. The first seed at which a source reaches a defect is one sample and not a rate. Timings come from one laptop and will differ elsewhere. The claims rest on exit status and error text, which do not vary between our runs.

*External threats.* We tested one engine, mined one other engine, and all four defects are in one family of string functions in one file. Three share a cause. Nothing here shows that the method finds defects in another engine, and Section 4.9 gives two reasons to doubt that it transfers cheaply: it needs a rich, labelled tracker, and most mined reports do not run on a different dialect. The comparison in Table 2 measures Graft as it stands now. The generator gained `lpad`, `rpad` and `repeat` only after the first defects were filed, so the table does not say which source would have found them first at that time.

*Construct threats.* We measure the blind spot through one table of functions, one literal policy and a text search of one source tree. That is a sound lower bound: a generator cannot emit a function it does not name or a value outside its configured range. It is not a proof that no other code path in the project reaches the same statement. We also count defects by signature. A signature is a hash of the error, so two causes can share one, as defects 1 and 2 do, and one cause can have two, as defect 1 does. For that reason we decided the count by reading the code, not from signatures. The blind-spot argument itself was formed after the first defect was found, as an explanation of why it had survived. We present it as a hypothesis that the evidence supports, not as the reason the first defect was found.

## 7 Conclusion

A fuzzer cannot test a function it never generates or judge one its reference engine lacks. Turso fuzzes itself with 41,205 lines of Rust in continuous integration. Yet `select LPAD('a', 9223372036854775807, 'b')` ended its process. None of its fuzzers ever calls `lpad`, their integers stop at $10^6$, and SQLite has no `lpad` to compare against.

We presented Graft, which reads an incumbent fuzzer's source to find its blind spot. It fills that region with the shapes of another engine's bug reports and with typed generation. It judges results with an oracle that needs no reference engine. It found three previously unknown defects in Turso and regenerates each from a recorded seed. Our patch for them exposed a fourth, which Graft did not find, and our two merged patches fix all four. At an equal budget of 20,000 cases, mined reports and typed generation each reached both root causes of the three defects, and drawing from both found the first defect at case 101. The fourth defect shows the method's own blind spot: boundary pools built from machine-integer limits miss the limits an engine sets itself.

## Data Availability

The replication package accompanies this submission [Anonymous 2026a]. It contains the source of Graft with its 514 tests; the mined DuckDB corpus with the mining run's own counters; one page per defect with its reproduction, measured before-and-after, root cause and, for defects 1 to 3, the regenerating seed; both patches with the issue and pull-request text as submitted; the raw results behind Table 2, including the repeat on the first-patch build; and the exact Turso commits used. The command `make reproduce` replays the chain in Figure 1 and, when both Turso builds are present, runs defects 1 and 4 before and after the patches.

## Use of Generative AI

A generative AI assistant, Claude by Anthropic, was used during this project for help with writing and reviewing code, analysing results, and drafting the upstream issue reports, the documentation and this paper. Its use is disclosed in the body of both issues filed with the Turso maintainers. The

authors checked every statement, every citation and every number. Every citation was verified against its DOI record, and all numbers reported here were produced by running the tool and the commands given in the replication package.

## References


Anonymous. 2026a. GRAFT: source code, mined corpus, defect reproductions, patches and experiment data. Replication package supplied with this submission. Location withheld for anonymous review.

Anonymous. 2026b. Issue #11219: LPAD/RPAD with an extremely large length aborts the process instead of returning a SQL error. https://github.com/tursodatabase/turso/issues/11219. Closed as completed, 2026-11-02. Accessed: 2026-11-10.

Anonymous. 2026c. Issue #11260: lpad/rpad with a multi-byte fill build the whole string before the byte limit rejects it. https://github.com/tursodatabase/turso/issues/11260. Closed as completed, 2026-11-02. Accessed: 2026-11-10.

Anonymous. 2026d. Pull request #11225: check the requested length before allocating in lpad/rpad/repeat. https://github.com/tursodatabase/turso/pull/11225. Merged 2026-11-02 as commit ec86571642a6. Accessed: 2026-11-10.

Anonymous. 2026e. Pull request #11259: check the byte length of a multi-byte pad before building it. https://github.com/tursodatabase/turso/pull/11259. Merged 2026-11-02 as commit 121cd0a8c80b. Accessed: 2026-11-10.

Bachir Bendrissou, Alastair F. Donaldson, and Cristian Cadar. 2026. Repair-Driven Greybox Fuzzing. *Proceedings of the ACM on Software Engineering* 3, ISSTA (2026). doi:10.1145/3832229

Alexander Bulekov, Bandan Das, Stefan Hajnoczi, and Manuel Egele. 2023. No Grammar, No Problem: Towards Fuzzing the Linux Kernel without System-Call Descriptions. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society. doi:10.14722/ndss.2023.24688

Liqing Cao, Haofeng Li, Chenghang Shi, Jie Lu, Haining Meng, Lian Li, and Jingling Xue. 2025a. VulPA: Detecting Semantically Recurring Vulnerabilities with Multi-object Typestate Analysis. *Proceedings of the ACM on Software Engineering* 2, FSE (2025). doi:10.1145/3729378

Yiheng Cao, Susheng Wu, Ruisi Wang, Bihuan Chen, Yiheng Huang, Chenhao Lu, Zhuotong Zhou, and Xin Peng. 2025b. Recurring Vulnerability Detection: How Far Are We? *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025). doi:10.1145/3728901

DuckDB. 2026. Issue #27846: lpad/rpad ~75% slower in 2.0.0 than in 1.5.5 (ASCII pad character). https://github.com/duckdb/duckdb/issues/27846. Closed 2026-10-27, labelled *reproduced*. Accessed: 2026-10-28.

Patrice Godefroid, Adam Kiezun, and Michael Y. Levin. 2008. Grammar-Based Whitebox Fuzzing. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 206–215. doi:10.1145/1375581.1375607

Samuel Groß, Simon Koch, Lukas Bernhard, Thorsten Holz, and Martin Johns. 2023. FUZZILLI: Fuzzing for JavaScript JIT Compiler Vulnerabilities. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society. doi:10.14722/ndss.2023.24290

Qixuan Guo and Yongzhong He. 2026. Accurate Identification of the Vulnerability-Introducing Commit based on Differential Analysis of Patching Patterns. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society. doi:10.14722/ndss.2026.230140

Lukas Kirschner and Ezekiel Soremekun. 2025. Directed Grammar-Based Test Generation. *IEEE Transactions on Software Engineering* 51, 12 (2025). doi:10.1109/TSE.2025.3627220

Yu Liang and Peng Liu. 2026. Efficiently Detecting DBMS Bugs through Bottom-up Syntax-based SQL Generation. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society. doi:10.14722/ndss.2026.240198

Manuel Rigger and Zhendong Su. 2020a. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 1140–1152. doi:10.1145/3368089.3409710

Manuel Rigger and Zhendong Su. 2020b. Finding Bugs in Database Systems via Query Partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020). doi:10.1145/3428279

Zixuan Tan, Jiayuan Zhou, Xing Hu, Shengyi Pan, Kui Liu, and Xin Xia. 2025. Similar but Patched Code Considered Harmful: The Impact of Similar but Patched Code on Recurring Vulnerability Detection and How to Remove Them. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE. doi:10.1109/ICSE55347.2025.00110

Turso Database. 2026a. Differential oracle: scalar function vocabulary and literal policy. https://github.com/tursodatabase/turso/tree/b588fbdc6c016cb3626d0ab464a7c45a54bc2b86/testing/differential-oracle/sql_gen/src. Files `functions.rs` and `policy.rs` at commit b588fbdc6. Accessed: 2026-11-11.

Turso Database. 2026b. Turso: an in-process SQL database, compatible with SQLite. https://github.com/tursodatabase/turso. Version 0.9.0-pre.2. Accessed: 2026-11-11.

Susheng Wu, Ruisi Wang, Yiheng Cao, Bihuan Chen, Zhuotong Zhou, Yiheng Huang, JunPeng Zhao, and Xin Peng. 2025. Mystique: Automated Vulnerability Patch Porting with Semantic and Syntactic-Enhanced LLM. *Proceedings of the ACM on Software Engineering* 2, FSE (2025). doi:10.1145/3715718

Haoyu Xiao, Yuan Zhang, Minghang Shen, Chaoyang Lin, Can Zhang, Shengli Liu, and Min Yang. 2024. Accurate and Efficient Recurring Vulnerability Detection for IoT Firmware. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 3317–3331. doi:10.1145/3658644.3670275

Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 955–970. doi:10.1145/3372297.3417260